

Automating Startup Matchmaking

Task 2: Policy Hacker test task, startin.lv | smalkaisk@gmail.com | 2026-09-15

0. Week one starts with people, not code

Before writing a single prompt, week one starts with a conversation, sitting down with whoever currently handles these requests and going through the last 15–20 real ones together: an event request and an investor request side by side, so we can actually see where the decisions differ. What we're really after is the stuff that never made it into the spreadsheet: is this startup actively raising money right now? Have they presented well before? Have we already suggested them to three other investors this month? A lot of that lives in someone's head, not in a column. Everything below builds on what that conversation turns up.

1. How it would actually work, step by step

We're assuming requests arrive as ordinary emails, since that's what the brief describes. We don't know yet whether startin.lv uses Gmail, Outlook, or something else. The plan works either way, and it's a detail to confirm during discovery, not something that changes the design.

1. **Filtering out the noise, cheaply:** most inboxes get plenty of email that has nothing to do with matchmaking. Before anything expensive happens, a simple keyword and sender check throws out the obvious non-matches for free, and anyone who's sent a real request before gets recognised automatically. Only the roughly 10 requests a week that actually look real go through the full, cheap AI check, not every email that lands in the inbox.
2. **Deciding what counts as a real request:** in week one, a person still makes that call, just by labelling the email, the same as they'd do anyway. It costs nothing, it's completely accurate, and it gives us real examples to teach the automatic version later.
3. **Reading the request:** Claude reads the email and pulls out whatever it actually says: the type of request, sector or stage, location, how much they'd invest ("ticket size", basically how big a cheque they usually write), any deadline. It reads Latvian or English equally well, so the language a request arrives in isn't a problem. For now it only reads the first message in a thread, not a longer back-and-forth (see point 7).
4. **Splitting by type:** event/open-call requests and investor requests are probably looking for different things, so from here the two get handled separately (point 2).
5. **Narrowing the list down:** the ~600 startups get filtered to a realistic shortlist based on the plain facts: sector, stage, and so on. We'd deliberately include a few startups that haven't been suggested in a while too, not just the obvious best matches, so good-but-overlooked companies still get a chance.
6. **Ranking:** Claude scores the shortlist against criteria specific to that type of request and writes one line explaining why each pick makes sense. Before anything is finalised, we apply a plain rule that lowers the score of startups already pitched a lot recently (point 5). That part isn't left to the AI's judgement.
7. **A draft, never an email that just goes out:** this produces a draft in whatever email program the team already uses, with the shortlist, the reasoning, and the scores attached. A person reviews it and hits send themselves (point 4). Nothing sends on its own.

2. Two different kinds of requests

Event organisers and investors are probably looking for different things, but here's the honest part: we don't actually know what investors screen for today. Open calls usually spell out what they want plainly, but an investor's real criteria (what stage, what they'd reject immediately, how much the cheque size really matters next to who else is investing) lives in someone's experience, not the spreadsheet. Writing down "stage and ticket size" as the answer right now would just be dressing up a guess as a decision. That's exactly what the conversations in point 0 are meant to settle first.

So instead of locking in assumptions, each type of request gets its own small, easy-to-edit set of criteria we can adjust once we actually know what belongs in it. Before trusting the investor rubric at all, we'd test it against those same historical requests from point 0, checking whether the AI's picks would have matched what was actually sent. If it turns out to be scoring on the wrong things, that's a quick edit, not a rebuild. The event and open-call side is safer to build from day one, since what they're looking for is usually stated outright.

Event outcomes are easy to observe directly (did the organiser pick from the list, did the startup show up), and that's ultimately what tells us whether the investor side's assumptions were ever right; investor outcomes need the manual check-in already described in point 4.

3. Tools, and what it would cost each month

\$15–40 / month all in, mostly the AI usage (see the breakdown below)

We'd build this in **n8n**, a tool that lets the whole thing be built visually, as a flowchart anyone can look at and follow, rather than a script only one person can read, which matters for a small team in case whoever built it isn't the one maintaining it later. Running it on our own small VPS (a virtual private server we rent by the month) also keeps the cost simple and predictable, even for steps that need to check something against all 600 rows of the spreadsheet.

- A small VPS to run n8n on: a rough estimate for now, likely somewhere around \$5–10 a month, with the actual provider still to be researched properly. Whichever one is chosen, it should be easy to resize later if more automations get added.
- Google Sheets instead of Excel, so the same spreadsheet the team already knows can be read automatically: free.
- Connecting to whichever email inbox they use: free, within normal limits.
- Anthropic's AI (Claude) for reading requests and ranking candidates: the main cost, but the cheap filtering step above means it's only doing real work on genuine requests, not skimming every email that comes in.

4. Keeping a person in the loop

There isn't a separate screen to check. The review happens in the same email drafts folder the team already uses, so there's nothing new to learn:

- Someone decides what counts as a real request in the first place, by labelling the email.
- Before anything is sent, a draft shows what the AI understood from the request, who it picked and why, and how often each pick has been suggested recently. A person edits it and sends it themselves. Nothing goes out on its own.
- If nothing scores well, the draft says so plainly, instead of quietly presenting a weak match as if it were a strong one.
- Whatever a person changes before sending gets logged automatically, at no extra effort, and reviewed every so often to see if the AI keeps missing the same thing.
- For investor requests specifically, someone checks in occasionally on whether anything actually came of it, since that's not something the system can see on its own.

5. Not always picking the same favourites

If every request is scored on its own, the system will naturally keep suggesting whichever startups happen to have the fullest, most impressive-looking spreadsheet entries, and the other ~590 barely get a look in. The fix isn't to ask the AI to "be fair"; it isn't reliable at holding itself to that kind of rule consistently. Instead, we'd keep a simple count of how often each startup has been suggested recently and subtract points for that before the final ranking is shown to anyone, a plain calculation. Worth also checking now and then whether the scoring itself leans toward one sector or pitch style regardless of how often something's been suggested, since the point-deduction rule only catches repetition, not that kind of skew.

There's a second, different problem worth naming on its own: a startup with a thin, half-filled-in entry will score badly regardless of how good the company actually is, simply because there's less to go on. That isn't favouritism; it's a data-quality gap, fixed by flagging incomplete entries for someone to fill in (point 6), not by changing how scoring works.

6. Keeping the list up to date

We wouldn't build anything that constantly crawls the internet. Once a month, a scheduled check looks at each startup's website or LinkedIn for signs of recent activity and flags anything that looks stale or thin for someone to review; being suggested for a match is itself a natural moment to double-check that entry right then. New startups (whether the team finds them or a founder submits themselves) go through the same kind of human check before they're added to the matching pool, at least to start with, so nothing fake or wrong slips in unnoticed. Adding startups automatically from public sources is worth considering later, once we trust the process, not a week-one feature.

7. Privacy, and what we're not solving yet

We'd host our own VPS in the EU (exact provider still to be chosen), and Anthropic's AI service is the only outside company that ever sees this data. That's worth being upfront about, since investor emails can contain fairly sensitive information about what they're looking to invest in. A couple of things we're deliberately leaving open rather than pretending they're solved: in week one we only read the first message of an email, not a longer back-and-forth, so a reply typed into the same thread later would need a person to pick it up manually; and if the AI service or the inbox connection goes down partway through, someone should get notified rather than a request silently going unanswered. We'd also track how long each request takes to answer, and how often a person has to change a draft before sending, so we actually know whether this is chipping away at the 7 hours a week this currently costs, rather than just assuming it is.

8. Week one vs. month three

	Week one	Month three
Getting started	A person notices the email and starts things off	Happens automatically the moment a matching email arrives
Spotting a real request	A person decides what counts as one	The AI makes that call on its own, trained on what happened in week one
Matching	Both types handled separately; event criteria solid, investor criteria still a first guess	Investor criteria refined from real outcomes; repeat-suggestion penalty and data-quality flags running
The spreadsheet	Moved over from Excel once; new startups added only after a person checks them	Monthly freshness checks run without anyone starting them; new startups still get a human check
Learning from mistakes	Not enough data yet to learn from	Every edit a person makes gets logged and reviewed monthly
Getting notified	Not needed, a person is already watching	A message pings the team the moment a draft is ready